Technical Documentation — Workshop 4 (TP4)

Real-Time Waiting Room with Flutter & Supabase

Student: Iheb Najjar | Class: DSI 31 | Course: Cross-Platform Development

1. Overview & Objectives

The objective of Workshop 4 (TP4) is to transform a local Flutter queue application into a cloud-backed, distributed real-time system using Supabase as the Backend-as-a-Service (BaaS). Instead of keeping queue tickets in ephemeral device memory, state is synchronized across multiple connected devices via PostgreSQL database tables and WebSocket change subscriptions.

- • Connect Flutter to Supabase without requiring custom backend servers.
- • Synchronize state across devices in real time with WebSocket broadcast subscriptions.
- • Enforce data security through PostgreSQL Row Level Security (RLS) policies.
- • Apply Test-Driven Development (TDD) with unit, mock, and widget tests.

2. System Architecture

The system follows a reactive distributed architecture where Supabase acts as the single source of truth:

Layer	Role & Implementation
Database Layer	PostgreSQL database table 'clients' with auto-generated UUIDs and timestamps.
Realtime Engine	WebSocket channel ('supabase_realtime') broadcasting INSERT & DELETE events.
State Management	QueueProvider extending ChangeNotifier, managing local state and stream subscriptions.
Presentation Layer	WaitingRoomScreen using Consumer<QueueProvider> for responsive UI updates.

3. Detailed Function & Class Breakdown

A. Data Model (lib/models/client.dart)

Encapsulates queue tickets with type-safe properties and defensive JSON parsing:

- • id (String): Unique UUID generated by PostgreSQL gen_random_uuid().
- • name (String): Client name entered in the waiting room queue.
- • createdAt (DateTime): Server timestamp ensuring First-In-First-Out (FIFO) queue order.
- • Client.fromMap(Map): Parses database records into strongly typed objects with null-safety.
- • toMap(): Converts a Client instance into a JSON-compatible map for database writes.

B. State Provider (lib/queue_provider.dart)

Extends ChangeNotifier to synchronize local UI with cloud updates:

- • clients (List<Client>): Read-only unmodifiable list exposed to the UI layer.
- • _fetchInitialClients(): Queries the Supabase 'clients' table ordered by created_at ascending.
- • _setupRealtimeSubscription(): Subscribes to PostgreSQL INSERT and DELETE events via WebSockets.
- • addClient(String name): Validates and inserts a new client record into Supabase.
- • removeClient(String id): Deletes a client ticket from the cloud database by UUID.
- • nextClient(): Dequeues the earliest client in line (FIFO queue logic).
- • QueueProvider.forTesting(): Specialized constructor for injecting mock clients in tests.

C. User Interface (lib/main.dart)

- • main(): Loads .env, initializes Supabase, and bootstraps ChangeNotifierProvider.
- • WaitingRoomScreen: Displays the text input field, 'Add' button, and waiting list.
- • Consumer<QueueProvider>: Dynamically rebuilds list cards with timestamps and delete buttons.
- • Next Client Button: Prominent green action button dequeuing the front of the line.

4. Supabase Setup & Row Level Security (RLS)

Row Level Security (RLS) ensures that public users can interact with the waiting room queue while protecting the integrity of the underlying database:

```
CREATE TABLE public.clients (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE public.clients ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow public select" ON public.clients FOR SELECT TO anon USING (true);
CREATE POLICY "Allow public insert" ON public.clients FOR INSERT TO anon WITH CHECK (true);
CREATE POLICY "Allow public delete" ON public.clients FOR DELETE TO anon USING (true);
```

```
ALTER PUBLICATION supabase_realtime ADD TABLE public.clients;
```

5. User Guide: How to Run and Test

1. 1. Install Dependencies: Run 'flutter pub get'.
2. 2. Environment Variables: Configure SUPABASE_URL and SUPABASE_ANON_KEY in .env.
3. 3. Run Automated Tests: Run 'flutter test' (All 10 tests pass).
4. 4. Multi-Device Demonstration: Open two terminals and run 'flutter run -d chrome' and 'flutter run -d edge'. Adding a client on Window 1 instantly appears on Window 2 without refreshing.

6. Automated Verification Results

- • 10/10 automated unit, mock, and widget tests passed successfully.
- • 0 warnings or errors reported by 'flutter analyze'.
- • Multi-device WebSocket synchronization validated with zero polling overhead.